→ Bash Scripting - ~ TechWorld With Nana

- We can use pieces of code to interact with computer similarly as we do with GUI, automating tedious tasks, fastening repetitive work, etc.

eg. creating 100 folders takes manually creating the 100 folders.
sh script of 1 line can do same in split second.

- On mac -
Bash is installed, but default is zsh.
hit 'bash' to use bash in mac.

* On linux, default is bash.
* In windows, use WSL.

→ If a engineer needs to check logs of a service every day, manually doing this, requires writing cmds one by one,

- Repeat same cmd over & over.
- Wastes time, even with 2-3 svc.
- No aggregate summary - scattered files
- Mentally track everything
- Ez to lose progress if interrupted.

→ Using a shell script, we can put all commands in single file, & executing it, executes all cmds sequentially.

- Logic defined once
- Run with sgl command
- No manual repetition
- Automate repetitive task
- Consistent Execution
- Acts as documentation
- Error Handling

- ~~test~~
- text file with linux cmds
- .sh extensn

→ Create using touch cmd

edit using vim, nano...

make executable using chmod +x <name>

→ File extension .sh added as convention, & to make easy for differentiating.

- if file doesn't have -sh extn, & executed. (given +x permission),

linux will execute them.

- Human-readable
- Ez to scan directory
- Syntax-highlight

→ cmds like touch, nano... are also executables.

- shells like bash, zsh, shell have file extensn as .sh.

(How does one know in which program is script written?)

→ Shebang - (AKA Hashbang) (#!)

Character seq. at very top of script file that tells OS which interpreter to use to run the file.

bash → #!/bin/bash

POSIX shell → #!/bin/sh

→ Formatting & Readability -

Normally → · no content
· cmd start & end
· spacing
· Hard to understand w/o script
· Not human-readable

· echo · cmd to print, what is being done.

· echo -e "\n" - Interpret backslashes as escapes.

→ Using abs. path -

Relative paths break easily. Use abs path to work anywhere.

→ Variables -

Better to create common used things like

LOG_DIRECTORY = "/Users/name/logs"

& accessing as:

find $LOG_DIRECTORY -name "*.log" -mtime -1

→ Arrays - 0-indexed

Not allowed to have spaces around "="

like

VAR = "Hello" ✗

VAR="Hello" ✓

ERROR_PATTERNS = ("ERROR", "FATAL", "CRITICAL")

accessing:

${ERROR_PATTERNS[0]}

Cmd Substitution-

LOG_FILE = $(find $LOG_DIR -name "*.log" -mtime -1)

→ Saves cmd o/p as a var.

→ Loops -

Exec set of cmds for all elmts.

```
for LOG_FILE in LOG_FILES do
    cmds
done
```

→ makes code cleaner, smaller & more understandable.

. for PATTERN in ${ERROR_PATTERNS[@]}; do

[@] → Arr subscript means each elmt stays its separate entity.

[*] → expand ~~on~~ all elmts as one string.

→ Writing to file -

FILE = $"/Users/name/logs/Report.txt"

echo "Hello World" > $FILE

> - overwrite pvs content every time cmd is exec.

>> - append every new o/p to existing file o/p.

Redirect^{n} operators.

We don't get o/p in terminal after this.

→ Conditionals-

```
ERROR_COUNT = $(grep -c "$PATTERN" "$LOG_FILE")
echo $ERROR_COUNT >> "$REPORT_FILE"
if ["$ERROR_COUNT" -gt 10]; then
   echo " Action Reqd. too many errors $PATTERN in $LOG_FILE
fi
```